

views::enumerate

Document #:	P2164R9
Date:	2022-12-07
Programming Language C++	
Audience:	LEWG, LWG
Reply-to:	Corentin Jabot < corentin.jabot@gmail.com >

A struct with 2 members, how hard can it be?!

Abstract

We propose a view `enumerate` whose value type is a struct with 2 members `index` and `value` representing respectively the position and value of the elements in the adapted range.

Revisions

R9

Wording fixes

- Introduce *range-with-movable-references* to ensure the reference type of the underlying type can be moved.
- Remove `enumerate_view::iterator::index_type` and use `difference_type` everywhere. LEWG felt strongly neutral about `index_type` in Kona and LWG did not like it.
- `operator<=>` returns `strong_ordering` (always operate on ints).
- Ensure constructors don't brace initialize.
- Fix `iter_swap`' noexcept specification.
- Add missing constraint on `enumerate_view::end()`.
- `iterator::base()` now returns a const reference and is not constrained
- Attempt at improving the awkward formatting.
- Other small editorial fixups.

R8

- remove the wording for `enumerate` result following SG9 consensus to use `std::tuple`.
- Qualify `std::move`

- Italicize `base_` and `simple-view`
- Unconditionally use `range_difference_type` as the `index_type`
- Model the sentinel wording on `elements_view` in order to support mixed comparison
- Add missing `explicit` on the one parameter constructor
- Improve the specification of the `begin/end` methods
- Add missing `enable_borrowed_range` specialization
- Greatly simplify the comparison operators
- Propose an `index` methods to avoid deferencing the underlying iterator when not needed. The `*` operator of the underlying range might be expensive, if the user only cares about the index, the `index` method could be used to query the index cheaply in all cases. It is trivial to specify, implement, and could be provided later at no cost. But I'm happy to do that change if it increases consensus.
- Other minor wording fixes.

R7

- This version asks LEWG to choose between `tuple` or `enumerate_result` as the reference and value types of `enumerate_view`. The approach presented in previous revisions of having a value type and a reference type of different types proved not workable. We need to pick one of the two options. Wording is provided for both.
- Add missing `iter_move` and `iter_swap` functions.
- Add the markup for freestanding
- Add feature test macro
- Formatting fixes.

R6

Wording changes:

- Add `enumerate_result` to the list of tuple-like types
- Because `enumerate_view::iterator::operator*` returns values, `enumerate_view::iterator` cannot be a `Cpp17ForwardIterator`. Change `iterator_category` and add `iterator_concept` accordingly.

R5

Instead of adding complexity to `enumerate_result`, we assume changes made by [P2165R2 \[1\]](#). [P2165R2 \[1\]](#) makes `pair` constructible from *pair-like* objects, and associative containers deduction guides work with ranges of *pair-like* objects. With these changes, `enumerate_result`

can remain a simple aggregate. We just need to implement the tuple protocol for it (get, tuple_element, tuple_size).

[P2165R2 \[1\]](#) ensures a common reference exists between `enumerate_result` and `std::tuple` as long as one exists between each element.

`count_type` is renamed to `index_type`. I am not sure why I ever chose `count_type` as the initial name.

R4

This revision is intended to illustrate the effort necessary to support named fields for `index` and `value`. In previous revisions, the value and reference types were identical, a regrettable blunder that made the wording and implementation efforts smaller than they are. `reference` and `value_type` types however needs to be different, if only to make the `ranges::to` presented in this very paper.

If that direction is acceptable, better wording will be provided to account for these new reference and `value_type` types.

This revision also gets rid of the `const index` value as LEWG strongly agreed that it was a terrible idea to begin with, one that would make composition with other views cumbersome.

R3

- Typos and minor wording improvements

R2, following mailing list reviews

- Make `value_type` different from `reference` to match other views
- Remove inconsistencies between the wording and the description
- Add relevant includes and namespaces to the examples

R1

- Fix the index type

Tony tables

Before	After
<pre> std::vector days{"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"}; int idx = 0; for(const auto & d : days) { print("{} {} \n", idx, d); idx++; } </pre>	<pre> #include <ranges> std::vector days{"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"}; for(const auto & [index, value] : std::views::enumerate(days)) { print("{} {} \n", index, value); } </pre>

Motivation

The impossibility to extract an index from a range-based for loop leads to the use of non-range-based for loops, or the introduction of a variable in the outer scope. This is both more verbose and error-prone: in the example above, the type of `idx` is incorrect.

`enumerate` is a library solution solving this problem, enabling the use of range-based for loops in more cases.

It also composes nicely with other range facilities: The following creates a map from a vector using the position of each element as key.

```
my_vector | views::enumerate | ranges::to<map>;
```

This feature exists in some form in Python, Rust, Go (backed into the language), and in many C++ libraries: `ranges-v3`, `folly`, `boost::ranges (indexed)`.

The existence of this feature or lack thereof is the subject of recurring StackOverflow questions.

Design

`std::tuple` vs aggregate with name members

Following the trend of using meaningful names instead of returning pairs or tuples, one option is to use a struct with named public members.

```

struct enumerate_result {
    count index;
    T value;
};

```

This design was previously discussed by LEWGI in Belfast in the context of [P1894R0](#) [4], and many people have expressed a desire for such struct with names.

```

std::vector<double> v;
enumerate(view) | to<std::vector>(); // std::vector<std::tuple<std::size_t, double>>.
enumerate(view) | to<std::map>();    // std::map<std::size_t, double>.

```

Why not just always return a tuple and rely on structure binding?

If a range reference type is convertible to the index type, it is error-prone whether one should write

```
for(auto && [value, index] : view | std::views::enumerate)
for(auto && [index, value] : view | std::views::enumerate)
```

Having named members avoids this issue. The feedback I keep getting is “we should use a struct if we can”. Which is consistent with previous LEWG guidelines to avoid using pair when a more meaningful type is possible.

Why use a tuple?

The drawback of using a distinct type is that

```
auto vec = enumerate(view) | ranges::to<std::vector>();
```

would produce a `vector<enumerate_result<std::size_t, range_value_t<decltype(view)>>>` where ideally, I think it should produce a tuple.

Why not `enumerate_result` as reference type and tuple as value type?

This was the approach proposed by the previous revision of the paper and my preferred solution. Best of both world. It only has a small drawback: it doesn't work.

Many algorithms end up requiring `invocable<F&, iter_value_t<I>&> && invocable<F&, iter_reference_t<I>>>` (where F is a function), which would require `std::tuple<std::size_t, Foo>` to be convertible to `enumerate_result<std::size_t, Foo>`.

In practice, this means that many algorithms are not utilisable if reference and values are not the same type

```
std::ranges::find(enumerate(/*...*/), [](auto const& p) { // constraints not satisfied.
    return /*...*/;
})
```

This is simply not acceptable.

Tomasz also observed that it would interact poorly with `as_const`.

```
for (auto const& p : enumerate(/*...*/)) {
    something(p.value); // OK
}

for (auto const& p : enumerate(/*...*/) | views::as_const) {
    something(p.value); // KO decltype(p) is tuple<std::size_t, const Foo>
}
```

Which is... not great. The unfortunate `invocable<F&, iter_value_t<I>&>` constraints exists as some algorithms (not `find`) may constructs value types out of the elements of the range.

Where do we go from here?

We need to choose between using `tuple` or `enumerate_result`, and that type would be used for both the value type and the reference type.

POLL: Does LEWG prefer using `enumerate_result` (Option 1) rather than a `tuple` (Option 2) as the value and reference type of `enumerate_view::iterator`?

The wording provides both options.

Why not `zip(iota, view)`?

Zippping the view with `iota` **does not actually work** (see also [P2214R0 \[3\]](#)), and a custom `index_view` would need to be used as the first range composed with `zip`, so a custom `enumerate_view` with appropriately named members is not adding a lot of work.

`enumerate` as presented here is slightly less work for the compiler, but both solutions generate similar assembly.

Performance

An optimizing compiler can generate the same machine code for `views::enumerate` as it would for an equivalent `for` loop. [Compiler Explorer](#) [Editor's note: This implementation is a prototype not fully reflective of the proposed design] .

Implementation

This proposal has been implemented ([Github](#)) There exist an implementation in `ranges-v3` (where the `enumerate_view` uses `zip_with` and a pair value type).

Proposal

We propose a view `enumerate` whose value type is a struct with 2 members `index` and `value` representing respectively the position and value of the elements in the adapted range.

Wording



Header `<ranges>` synopsis

[[ranges.syn](#)]

```
template<class R>
using keys_view = elements_view<R, 0>; // freestanding
template<class R>
using values_view = elements_view<R, 1>; // freestanding

namespace views {
```

```

    template<size_t N>
    inline constexpr unspecified elements = unspecified;    // freestanding
    inline constexpr auto keys = elements<0>;                // freestanding
    inline constexpr auto values = elements<1>;                // freestanding
}

template<input_range View>
requires view<View>
class enumerate_view; // freestanding

template<class View>
inline constexpr bool enable_borrowed_range<enumerate_view<View>> =    // freestanding
enable_borrowed_range<View>;

namespace views { inline constexpr unspecified enumerate = unspecified; } // freestanding

// ??, zip view
template<input_range... Views>
requires (view<Views> && ...) && (sizeof...(Views) > 0)
class zip_view; // freestanding

template<class... Views>
inline constexpr bool enable_borrowed_range<zip_view<Views...>> = // freestanding
(enable_borrowed_range<Views> && ...);

namespace views { inline constexpr unspecified zip = unspecified; } // freestanding

[...]

}

```

❖ Helper concepts

[range.utility.helpers]

Many of the types in subclause ?? are specified in terms of the following exposition-only concepts:

```

template<class R>
concept simple-view = // exposition only
view<R> && range<const R> &&
same_as<iterator_t<R>, iterator_t<const R>> &&
same_as<sentinel_t<R>, sentinel_t<const R>>;

template<class I>
concept has-arrow = // exposition only
input_iterator<I> && (is_pointer_v<I> || requires(I i) { i.operator->(); });

template<class T, class U>
concept different-from = // exposition only
!same_as<remove_cvref_t<T>, remove_cvref_t<U>>;

```

[Editor's note: The intent is that this concept can be reused by LWG3731 [2]]

```
template <class R>
concept range-with-movable-references = // exposition only
input_range<R> && move_constructible<range_reference_t<R>> &&
move_constructible<range_rvalue_reference_t<R>>;
```

[Editor's note: Add the following new section before [range.zip]]

 **Enumerate view** [range.enumerate]

 **Overview** [range.enumerate.overview]

`enumerate_view` is a view whose elements represent both the position and value from a sequence of elements.

The name `views::enumerate` denotes a range adaptor object. Given a subexpression `E`, the expression `views::enumerate(E)` is expression-equivalent to `enumerate_view<views::all_t<decltype(E)>>(E)`.

[Example:

```
vector<int> vec{ 1, 2, 3 };
for (auto [index, value] : enumerate(vec) )
    cout << index << ":" << value << ' '; // prints: 0:1 1:2 2:3
```

— end example]

 **Class template `enumerate_view`** [range.enumerate.view]

```
template<view V>
requires range-with-movable-references<V>
class enumerate_view : public view_interface<enumerate_view<V>> {

    V base_ = V(); // exposition only

    template<bool Const>
    class iterator; // exposition only
    template<bool Const>
    class sentinel; // exposition only

public:

    constexpr enumerate_view() requires default_initializable<V> = default;
    constexpr explicit enumerate_view(V base);

    constexpr auto begin() requires (!simple-view<V>)
    { return iterator<false>(ranges::begin(base_), 0); }

    constexpr auto begin() const requires range-with-movable-references<const V>
    { return iterator<true>(ranges::begin(base_), 0); }
```



```

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (common_range<V> && sized_range<V>)
        return iterator<false>(ranges::end(base_), ranges::distance(base_));
    else
        return sentinel<false>(ranges::end(base_));
}

constexpr auto end() const requires range-with-movable-references<const V> {
    if constexpr (common_range<const V> && sized_range<const V>)
        return iterator<true>(ranges::end(base_), ranges::distance(base_));
    else
        return sentinel<true>(ranges::end(base_));
}

constexpr auto size()
requires sized_range<V>
{ return ranges::size(base_); }

constexpr auto size() const
requires sized_range<const V>
{ return ranges::size(base_); }

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }
};
template<class R>
enumerate_view(R&&) -> enumerate_view<views::all_t<R>>;
}

```

constexpr explicit enumerate_view(V base);

Effects: Initializes *base_* with `std::move(base)`.

Class `enumerate_view::iterator` [range.enumerate.iterator]

```

namespace std::ranges {
    template<view V>
    requires range-with-movable-references<V>
    template<bool Const>
    class enumerate_view<V>::iterator {
        using Base = maybe_const<Const, V>; // exposition only
    public:
        using iterator_category = input_iterator_tag;
        using iterator_concept = see_below;
        using difference_type = range_difference_t<Base>;
        using value_type = tuple<difference_type, range_value_t<Base>>;

    private:
        using reference_type = // exposition only

```

```

        tuple<difference_type, range_reference_t<Base>>;
        iterator_t<Base> current_ = iterator_t<Base>(); // exposition only
        difference_type pos_ = 0; // exposition only

        constexpr explicit
        iterator(iterator_t<Base> current,
                difference_type pos); // exposition only

public:

        iterator() requires default_initializable<iterator_t<Base>> = default;

        constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

        constexpr const iterator_t<Base> & base() const & noexcept;
        constexpr iterator_t<Base> base() &&;

        constexpr difference_type index() const noexcept;

        constexpr auto operator*() const {
            return reference-type(pos_, *current_);
        }

        constexpr iterator& operator++();
        constexpr void operator++(int);
        constexpr iterator operator++(int) requires forward_range<Base>;

        constexpr iterator& operator--() requires bidirectional_range<Base>;
        constexpr iterator operator--(int) requires bidirectional_range<Base>;

        constexpr iterator& operator+=(difference_type x)
        requires random_access_range<Base>;
        constexpr iterator& operator-=(difference_type x)
        requires random_access_range<Base>;

        constexpr auto operator[](difference_type n) const
        requires random_access_range<Base>
        { return reference-type(pos_ + n, current_[n]); }

        friend constexpr bool operator==(const iterator& x, const iterator& y) noexcept;
        friend constexpr strong_ordering operator<=>(const iterator& x, const iterator& y)
        noexcept;

        friend constexpr iterator operator+(const iterator& x, difference_type y)
        requires random_access_range<Base>;
        friend constexpr iterator operator+(difference_type x, const iterator& y)
        requires random_access_range<Base>;
        friend constexpr iterator operator-(const iterator& x, difference_type y)
        requires random_access_range<Base>;
        friend constexpr difference_type operator-(const iterator& x, const iterator& y);

```

```

        friend constexpr auto iter_move(const iterator& i)
        noexcept(noexcept(ranges::iter_move(i.current_))
            && is_nothrow_move_constructible_v<range_rvalue_reference_t<Base>>) {
            return tuple<difference_type,
                range_rvalue_reference_t<Base>>(pos_, ranges::iter_move(i.current_));
        }
    };
}

```

The member *typedef-name* `iterator::iterator_concept` is defined as follows:

- If *Base* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.
- Otherwise, if *Base* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- Otherwise, if *Base* models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- Otherwise, `iterator_concept` denotes `input_iterator_tag`.

```
constexpr explicit iterator(iterator_t<Base> current, difference_type pos = 0);
```

Effects: Initializes `current_` with `std::move(current)` and `pos_` with `pos`.

```
constexpr iterator(iterator<!Const> i)
requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

Effects: Initializes `current_` with `std::move(i.current_)` and `pos_` with `i.pos_`.

```
constexpr const iterator_t<Base> & base() const & noexcept;
```

Effects: Equivalent to: return `current_`;

```
constexpr iterator_t<Base> base() &&;
```

Effects: Equivalent to: return `std::move(current_)`;

```
constexpr difference_type index() const noexcept;
```

Effects: Equivalent to: return `pos_`;

```
constexpr iterator& operator++();
```

Effects: Equivalent to:

```

    ++current_;
    ++pos_;
    return *this;

```

```
constexpr void operator++(int);
```

Effects: Equivalent to:

```
++*this;
```

constexpr *iterator* operator++(int) requires forward_range<*Base*>;

Effects: Equivalent to:

```
auto temp = *this;
++*this;
return temp;
```

constexpr *iterator*& operator--() requires bidirectional_range<*Base*>;

Effects: Equivalent to:

```
--current_;
--pos_;
return *this;
```

constexpr *iterator* operator--(int) requires bidirectional_range<*Base*>;

Effects: Equivalent to:

```
auto temp = *this;
--*this;
return temp;
```

constexpr *iterator*& operator+=(difference_type n)
requires random_access_range<*Base*>;

Effects: Equivalent to:

```
current_ += n;
pos_ += n;
return *this;
```

constexpr *iterator*& operator-=(difference_type n)
requires random_access_range<*Base*>;

Effects: Equivalent to:

```
current_ -= n;
pos_ -= n;
return *this;
```

friend constexpr bool operator==(const *iterator*& x, const *iterator*& y) noexcept;

Effects: Equivalent to: `return x.pos_ == y.pos_;`

```
friend constexpr strong_ordering operator<=>(const iterator& x, const iterator& y)
noexcept;
```

Effects: Equivalent to: `return x.pos_ <=> y.pos_;`

```
friend constexpr iterator operator+(const iterator& x, difference_type y)
requires random_access_range<Base>;
```

Effects: Equivalent to:

```
    auto temp = x;
    temp += y;
    return temp;
```

```
friend constexpr iterator operator+(difference_type x, const iterator& y)
requires random_access_range<Base>;
```

Effects: Equivalent to: `return y + x;`

```
constexpr iterator operator-(const iterator& x, difference_type y)
requires random_access_range<Base>;
```

Effects: Equivalent to:

```
    auto temp = x;
    temp -= y;
    return temp;
```

```
constexpr difference_type operator-(const iterator& x, const iterator& y);
```

Effects: Equivalent to: `return x.pos_ - y.pos_;`



Class template `enumerate_view::sentinel`

[range.enumerate.sentinel]

```
namespace std::ranges {
    template<view V>
    requires range_with_movable_references<V>
    template<bool Const>
    class enumerate_view<V>::sentinel { // exposition only
        using Base = maybe_const<Const, V>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only
        constexpr explicit sentinel(sentinel_t<Base> end);
    public:
        sentinel() = default;
        constexpr sentinel(sentinel_t<!Const> other)
        requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;
```

```

template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
operator-(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
operator-(const sentinel& x, const iterator<OtherConst>& y);
};
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

Effects: Initializes *end_* with `std::move(end)`.

```
constexpr sentinel(sentinel<!Const> other)
requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

Effects: Initializes *end_* with `std::move(other.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

Effects: Equivalent to: `return end_;`

```
template<bool OtherConst>
requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

Effects: Equivalent to: `return x.current_ == y.end_;`

```
template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
operator-(const iterator<OtherConst>& x, const sentinel& y);
```

Effects: Equivalent to: `return x.current_ - y.end_;`

```
template<bool OtherConst>
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
operator-(const sentinel& x, const iterator<OtherConst>& y);
```

Effects: Equivalent to: `return x.end_ - y.current_;`

Feature test macro

[Editor's note: define `__cpp_lib_ranges_enumerate` set to the date of adoption in `<version>` and `<ranges>`].

Acknowledgments

Thanks a lot to Tomasz Kamiński for finding defects in the design proposed in earlier revisions, as well as his invaluable wording feedbacks. Thanks to Barry Revzin and Christopher Di Bella for their inputs on the design.

References

- [1] Corentin Jabot. P2165R2: Compatibility between tuple, pair and tuple-like objects. <https://wg21.link/p2165r2>, 6 2021.
- [2] Hewill Kang. LWG3731: zip_view and adjacent_view are underconstrained. <https://wg21.link/lwg3731>.
- [3] Barry Revzin, Conor Hoekstra, and Tim Song. P2214R0: A plan for c++23 ranges. <https://wg21.link/p2214r0>, 10 2020.
- [4] Andrew Tomazos. P1894R0: Proposal of std::upto, std::indices and std::enumerate. <https://wg21.link/p1894r0>, 10 2019.
- [N4885] Thomas Köppe *Working Draft, Standard for Programming Language C++* <https://wg21.link/N4885>